PROJECT: REFACTORING AND DELIVERY

Каталог фільмів
з адаптивними рекомендаціями

Розділ 4. Refactoring. Preparation for deployment & delivery

Роботу виконала студентка
1 курсу магістратури
комп’ютерних наук
Група інформатика -1
Голубцова Орина Євгенівна

Київ – 2026

4.1 Making changes to the system. Refactoring

Рефакторинг виконано без зміни функціональних вимог і формули рекомендацій. Мета - прибрати дублювання, розділити відповідальності, прискорити повторні запити та спростити підтримку.

Техніка	Компонент	Результат
Extract Method	refresh_rating_aggregate	Один перерахунок агрегованого рейтингу після оцінки або видалення профілю.
Extract Module	catalog.py	Пошук, пагінацію та кеш сторінок винесено з HTTP-маршрутів.
Introduce Cache	catalog/recommendation	Обмежений кеш 256 сторінок і готових Top-K; контрольована інвалідація.
Split component	main.tsx / App.tsx	Точку запуску відокремлено від стану та логіки інтерфейсу.
Consolidate API	/me/movie-state/{id}	Одна відповідь для оцінки й станів Favorite/Watchlist/Watched.
Responsive refactor	mobile navigation	Явне меню «Ще», читабельна історія, відсутність horizontal scroll.

Приклад до і після

До: однакова логіка перерахунку рейтингу в кількох маршрутах.
Після: refresh_rating_aggregate(db, movie_id) викликається з усіх шляхів зміни оцінок.

До: route /movies містив фільтрацію, сортування і pagination.
Після: route викликає search_movies(...), а catalog.py керує запитом і bounded cache.



Рисунок 4.1 – Regression testing після рефакторингу: 17 тестів пройдено

Після змін повторно виконано pytest, branch coverage, production build, Alembic check і перевірку 30/30 записів. Функціональна поведінка збережена.

4.2 Functional testing

Функціональні сценарії побудовано за ролями Guest, User та Administrator. Тестування проводилось на рівні API і реального браузера.

ID	Роль і сценарій	Очікуваний / фактичний результат
FT1	Guest: пошук і перегляд картки	Каталог доступний; фільтри й pagination працюють - Passed
FT2	User: login → Watchlist → rating → recommendations	Стан збережено, рекомендації оновлено - Passed
FT3	User: email/password/delete account	Перевірки пароля й унікальності, cascade cleanup - Passed
FT4	Admin: genre/movie CRUD і metadata sync	Role check і повні поля фільму - Passed
FT5	Expired/broken token та User проти admin API	401/403, захищені дані не повертаються - Passed
FT6	Mobile: навігація, картка, списки, історія	Без horizontal scroll; основні дії доступні - Passed

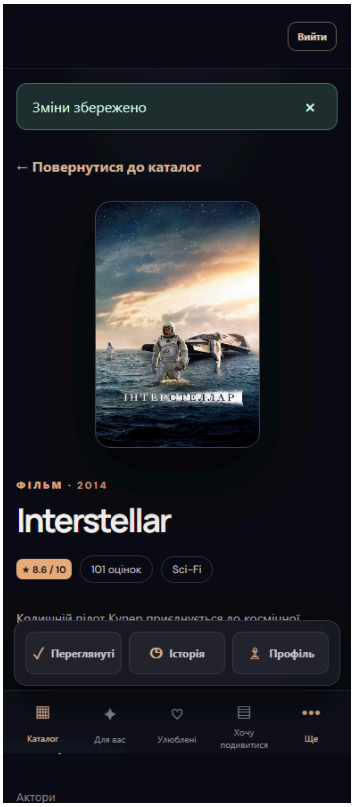


Рисунок 4.2 – Мобільний функціональний сценарій користувача

Контрольна перевірка	Результат
Automated browser sessions	5/5 технічних сесій успішні
Catalog completeness	30/30 повних записів
Frontend build	Успішно, 29 модулів
Backend regression	17 passed, 0 failed
Human usability NFR4.4	Не проводилось через відсутність 5 реальних учасників

4.3 Creating the package for deployment

Проєкт передається як вихідний код із відтворюваними локальним і контейнерним способами запуску. Docker Compose об’єднує PostgreSQL 16, FastAPI backend та React/nginx frontend. Backend автоматично застосовує Alembic migrations і виконує idempotent seed.

Артефакт	Призначення
docker-compose.yml	Оркестрація database, backend і frontend
backend/Dockerfile	Python 3.12, залежності, migration, seed, uvicorn

Артефакт	Призначення
frontend/Dockerfile + nginx.conf	Production build і reverse proxy /api
.env.example	Шаблон DATABASE_URL, JWT_SECRET, admin credentials і CORS
start-local.ps1 / stop-local.ps1	Локальна демонстрація без Docker

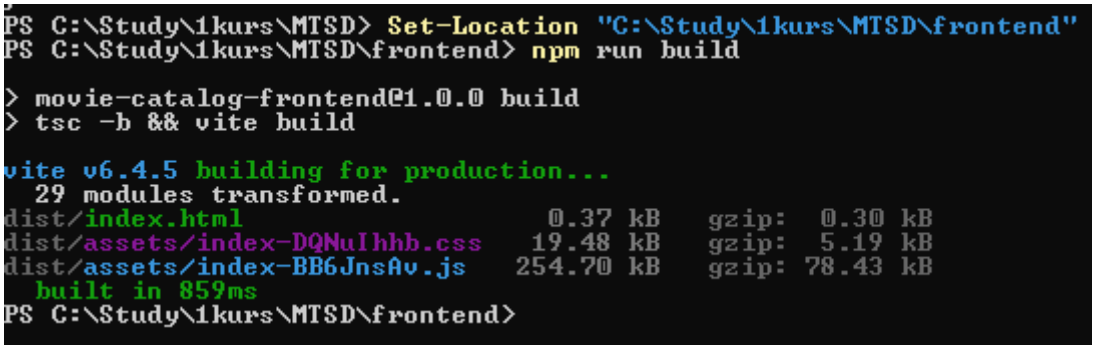


Рисунок 4.3 – Результат формування production-збірки frontend

Production-збірку React/TypeScript сформовано успішно; Vite завершив обробку 29 модулів без помилок.

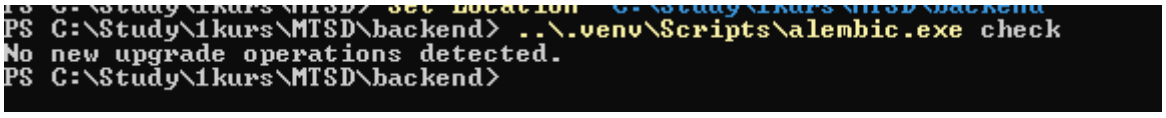


Рисунок 4.4 – Перевірка актуальності міграцій бази даних

Команда alembic check не виявила нових операцій оновлення, тому схема бази даних узгоджена з моделями застосунку.

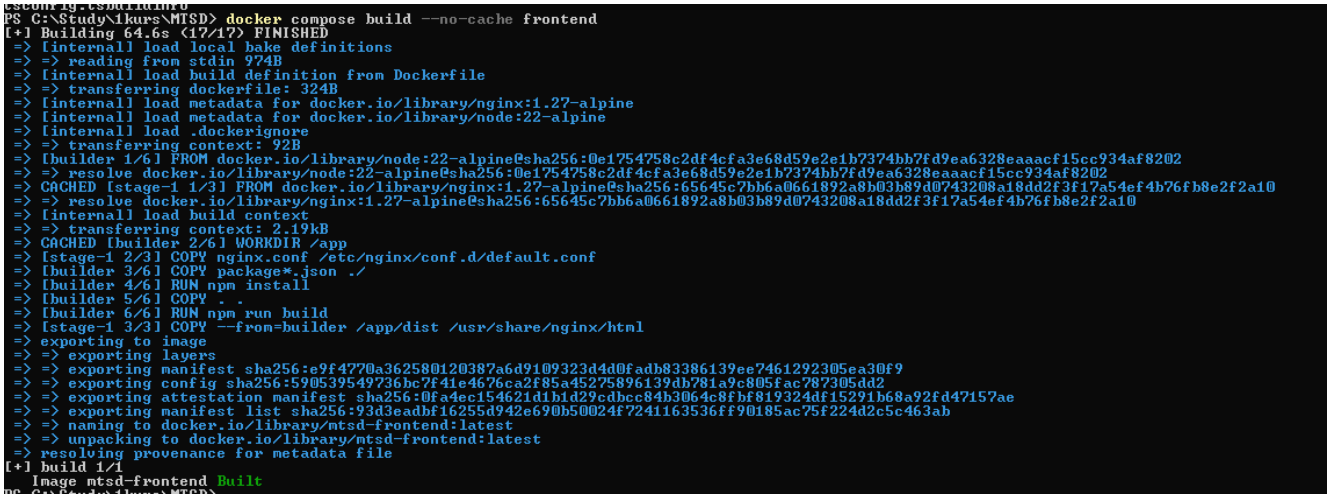


Рисунок 4.5 – Успішне формування Docker-образу frontend

Після додавання frontend/.dockerignore Docker BuildKit успішно зібрав production-образ із nginx.

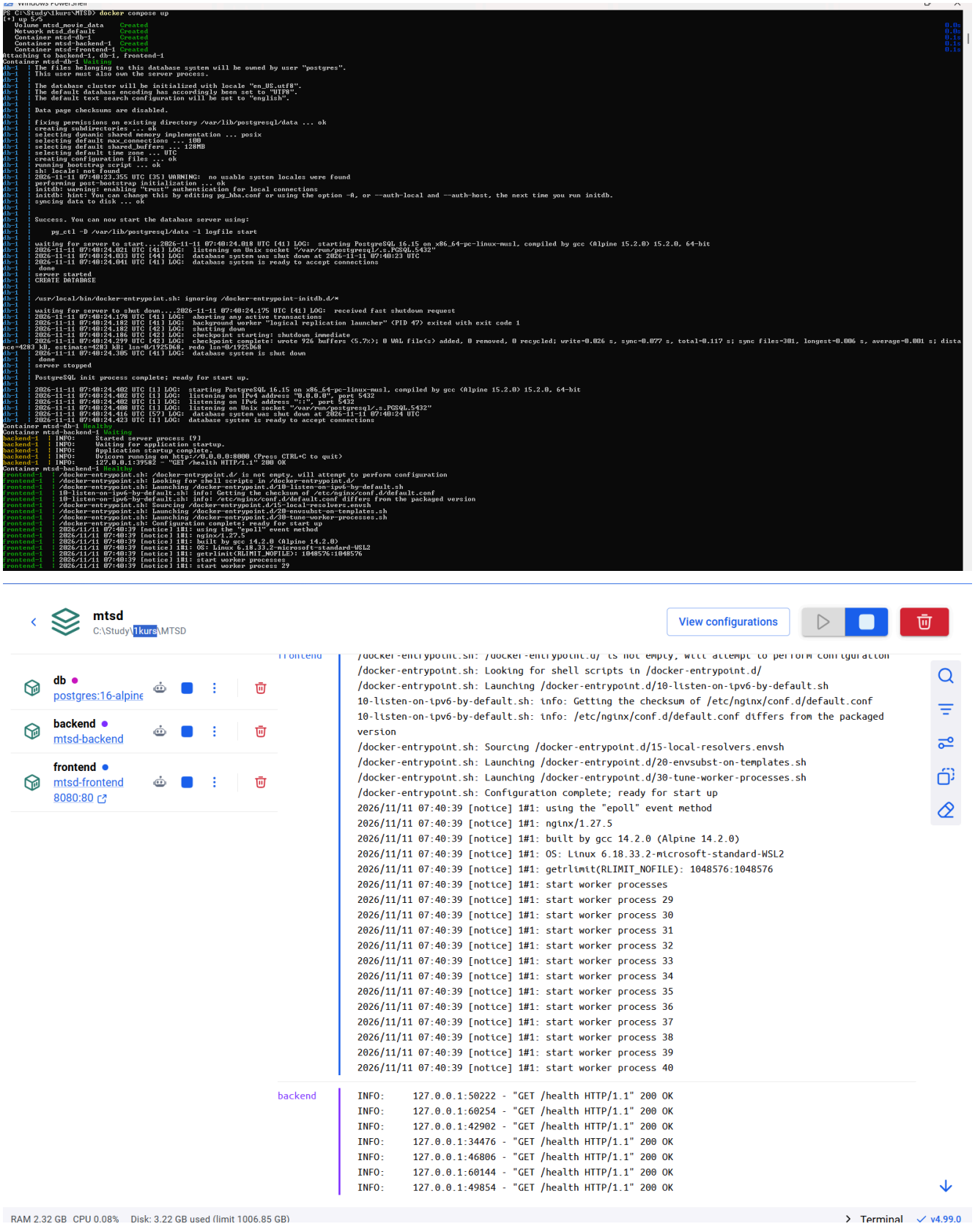


Рисунок 4.6 – Запущені контейнери застосунку в Docker Desktop

Контейнери PostgreSQL, backend і frontend запущено локально; health-запити backend повертають HTTP 200.

Контейнерний запуск

Контейнерне розгортання виконується через Docker Compose, який запускає PostgreSQL 16, FastAPI backend і React/nginx frontend як узгоджений набір сервісів.

Локальний запуск

Для локальної демонстрації використовуються скрипти start-local.ps1 і stop-local.ps1; API та OpenAPI-документація доступні на порту 8000.

Для публічного розгортання потрібно налаштувати домен, HTTPS/TLS 1.2+, окреме керування секретами, backup PostgreSQL та production CORS. Локальна Docker-конфігурація відкриває HTTP лише для демонстрації.

4.4 Measurement of resource capacity and performance

Після введення bounded cache повторено 30-хвилинний тест. Порівняно з базовим запуском P95 пошуку зменшився з 1197,90 до 855,86 мс, а P95 рекомендацій - з 1712,28 до 870,67 мс.

Показник	До рефакторингу	Після	Ціль
Search P95	1197,90 мс	855,86 мс	≤ 800 мс - не досягнуто
Recommendation P95	1712,28 мс	870,67 мс	≤ 1200 мс - досягнуто
HTTP 5xx	0	0	< 1% - досягнуто
Persistence mismatches	-	0 (80/80)	0 - досягнуто
Continuous duration	1800 с	1800,05 с	≥ 1800 с - досягнуто

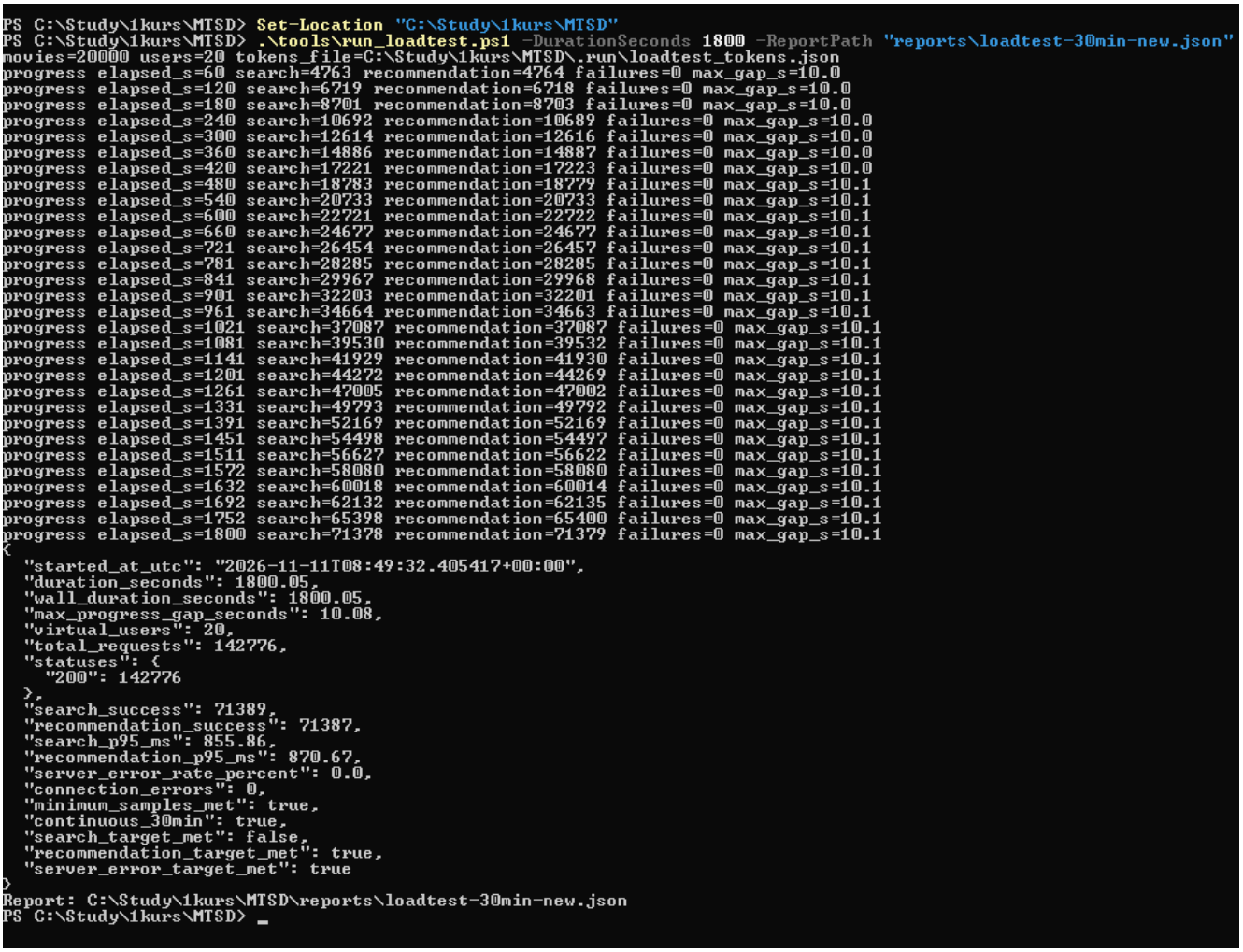


Рисунок 4.7 – Результат вимірювання після рефакторингу

Компонент	Мінімальна конфігурація для цільової перевірки
CPU	2 vCPU
RAM	4 GB
Database	PostgreSQL 16, до 20 000 фільмів
Load	20 virtual users, не менше 1000 запитів кожного типу

Після рефакторингу P95 пошуку становить 855,86 мс, а P95 формування рекомендацій — 870,67 мс. Запити рекомендацій відповідають установленій цілі, а пошук потребує подальшої оптимізації. Протягом 30 хвилин серверних і мережевих помилок не зафіксовано.

Поточні цифри отримано на Windows/SQLite без точного обмеження RAM до 4 GB. Тому остаточний висновок про NFR4.1 робиться лише після повторення на цільовій інфраструктурі.

4.5 API documentation and user guide

FastAPI автоматично формує OpenAPI 3.1. Інтерактивна документація доступна локально за /docs, а в Docker - за /api/docs. Через Authorize можна перевіряти захищені маршрути з JWT.



Рисунок 4.8 – Інтерактивна OpenAPI-документація Movie Catalog API

Група	Основні маршрути	Доступ
Auth	/auth/register, /auth/login, /auth/logout, /auth/me	Public / JWT
Catalog	/genres, /movies, /movies/{id}, /movies/{id}/similar	Public
Personal	/me/ratings, /me/{list}, /me/history, /me/recommendations	User JWT
Profile	/me/email, /me/password, /me/account	User JWT
Admin	/admin/genres, /admin/movies, /admin/sync	Admin JWT

Коротка інструкція користувача

- відкрити адресу frontend і вибрати «Увійти» або «Зареєструватися» у верхній частині;
- знайти фільм пошуком або фільтрами та відкрити його картку;
- поставити оцінку 1–5, додати фільм до Улюблених/Watchlist або позначити Переглянутим;
- відкрити «Для вас», щоб побачити персональні Top-10 і пояснення;
- для адміністратора відкрити «Адмін» і керувати жанрами, фільмами або синхронізацією.

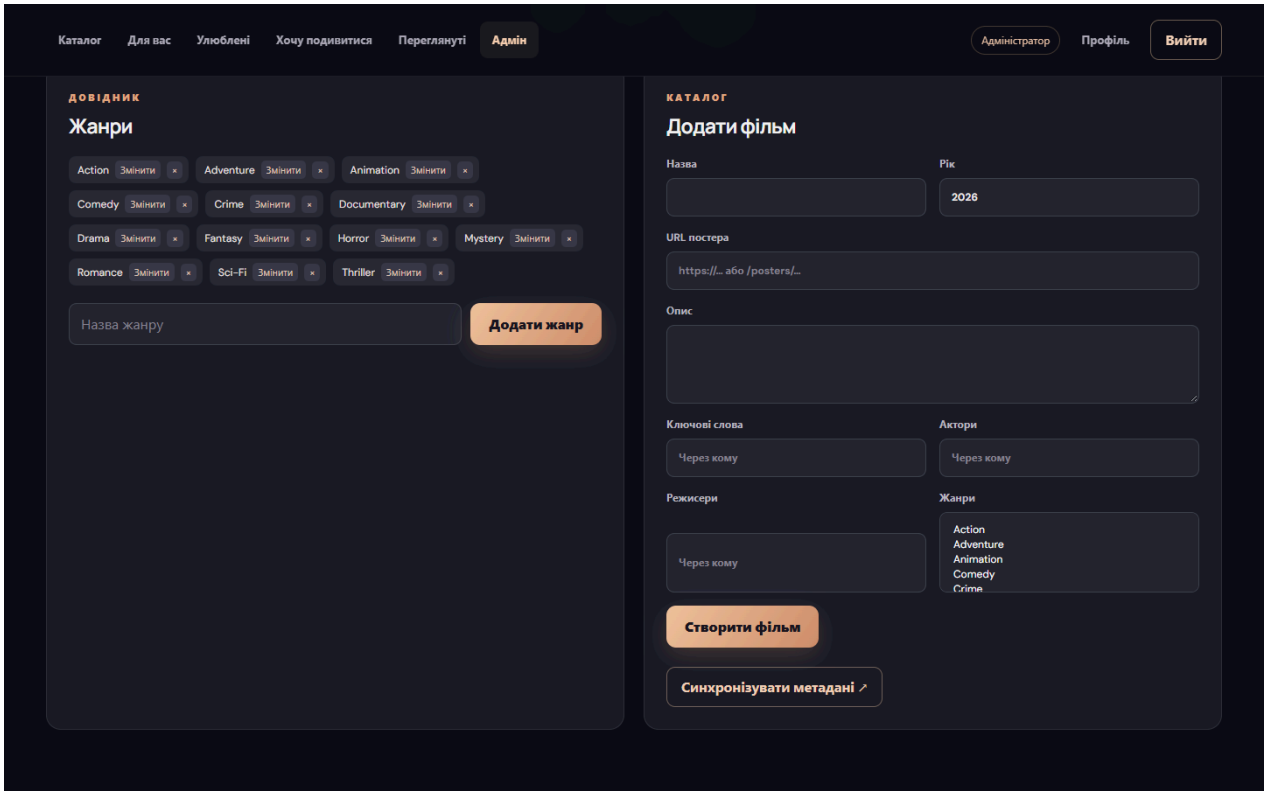


Рисунок 4.9 – Адміністративна форма з повними полями фільму

4.5.1 Verification of documentation and delivery

Перевірка	Спосіб підтвердження	Результат
API routes	OpenAPI /docs і зіставлення з main.py	Підтверджено
Database schema	alembic check	No new operations
Automated tests	pytest + coverage	17 passed; total 94%
Frontend package	npm run build	Успішно
Catalog content	verify_catalog_data.py	30/30

Перевірка	Спосіб підтвердження	Результат
Local delivery	start-local.ps1 / stop-local.ps1	Підтверджено
Docker delivery	docker-compose.yml і Dockerfiles	Підтверджено локально
Human usability	5 реальних учасників	Не перевірено
Public HTTPS	Production domain/TLS	Не входить у local demo

4.6 Conclusion

Рефакторинг зменшив дублювання й покращив P95. Проєкт має локальний та Docker-способи запуску, міграції, seed, OpenAPI, користувацькі сценарії й перевірений build. Для повного production sign-off потрібні PostgreSQL performance run, HTTPS-розгортання та usability-тест із реальними учасниками.